

Отчёт по лабораторной работе №23

по курсу «Алгоритмы и структуры данных».

Выполнил студент группы М8О-114БВ-24 Кожин Александр Сергеевич № по списку 7.

Контакты: mafioznca@gmail.com

Работа выполнена: «9» марта 2026 г.

Преподаватель: каф.806 Сысоев Максим Алексеевич

Входной контроль знаний с оценкой: _____

Отчет сдан «9» марта 2026 г.

Итоговая оценка: _____

Подпись преподавателя: _____

1. **Тема:** Составить программу на языке Си для построения и обработки дерева общего вида или упорядоченного двоичного дерева, содержащего узлы типа float, int, char или enum (для групп 1,8; 3,4; 2,5; 6,7 соответственно).
2. **Цель работы:** Проверить, находятся ли во всех листьях двоичного дерева элементы со значениями в заданном диапазоне.
3. **Задание:** Проверить, находятся ли во всех листьях двоичного дерева элементы со значениями в заданном диапазоне.
4. **Оборудование:** *Оборудование ПЭВМ студента, если использовалось:*
Процессор: Intel(R) Pentium(R) CPU 6405U @ 2.40GHz 2.40 GHz, ОП 8 ГБ, SSD 256 ГБ
5. **Программное обеспечение:** *Программное обеспечение ПЭВМ студента, если использовалось:*
Операционная система семейства Unix, наименование Linux Mint, версия 24.04 LTS, интерпретатор команд bash версия 5.2.21.
6. **Идея, метод, алгоритм решения задачи [в формах: словесной, псевдокода, графической (блок-схема, диаграмма, рисунок, таблица) или формальные спецификации с пред- и постусловиями]:**
 1. Проектирование структуры данных
 2. Построение упорядоченного дерева
 3. Алгоритм удаления узла
 4. Визуализация дерева
 5. Алгоритм функции, проверяющей, находятся ли все листья в заданном диапазоне [min, max]:
 6. Управление памятью
7. **Сценарий выполнения работы (план работы, первоначальный текст программы в черновике (можно на отдельном листе) и тесты либо соображения по тестированию).**

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <locale.h>
```

```
#include <windows.h>
```

```
#include <stdbool.h>
```

```
// Перечисление типов самолетов
```

```
typedef enum {
    CESSNA = 1,    // Легкомоторный самолет
    BEEHCRAFT,    // Частный самолет
    ANTONOV,       // Транспортный самолет
    SUKHOI,        // Истребитель
    AIRBUS,        // Пассажирский средний самолет
    BOEING,        // Пассажирский тяжелый самолет
    CONCORDE       // Сверхзвуковой самолет
} AircraftType;
```

// Структура узла двоичного дерева

```
typedef struct AirNode {
    AircraftType model;
    struct AirNode *leftWing;
    struct AirNode *rightWing;
} AirNode;
```

```
const char* getPlaneName(AircraftType t) {
    switch(t) {
        case AIRBUS:    return "Airbus";
        case BEEHCRAFT: return "Beechcraft";
        case ANTONOV:   return "Antonov";
        case SUKHOI:    return "Sukhoi";
        case: CESSNA:   return "Cessna";
        case BOEING:    return "Boeing";
        case CONCORDE:  return "Concorde";
        default:        return "Unknown Plane";
    }
}
```

// Создание нового узла

```
AirNode* createAirNode(AircraftType val) {
    AirNode* node = (AirNode*)malloc(sizeof(AirNode));
    if (!node) return NULL;
    node->model = val;
    node->leftWing = NULL;
    node->rightWing = NULL;
    return node;
}
```

```

// Добавляем узел и сохраняем порядок
AirNode* insertPlane(AirNode* root, AircraftType val) {
    if (!root) return createAirNode(val);
    if (val < root->model)
        root->leftWing = insertPlane(root->leftWing, val);
    else if (val > root->model)
        root->rightWing = insertPlane(root->rightWing, val);
    return root;
}

// Поиск минимального узла для удаления
AirNode* findMinPlane(AirNode* root) {
    while (root && root->leftWing) root = root->leftWing;
    return root;
}

// Удаление узла с освобождением памяти
AirNode* removePlane(AirNode* root, AircraftType val) {
    if (!root) return NULL;
    if (val < root->model)
        root->leftWing = removePlane(root->leftWing, val);
    else if (val > root->model)
        root->rightWing = removePlane(root->rightWing, val);
    else {
        if (!root->leftWing) {
            AirNode* temp = root->rightWing;
            free(root);
            return temp;
        } else if (!root->rightWing) {
            AirNode* temp = root->leftWing;
            free(root);
            return temp;
        }
        AirNode* temp = findMinPlane(root->rightWing);
        root->model = temp->model;
        root->rightWing = removePlane(root->rightWing, temp->model);
    }
    return root;
}

```

```
}
```

```
// Визуализация дерева с отступами
```

```
void printAirTree(AirNode* root, int depth) {
```

```
    if (!root) return;
```

```
    printAirTree(root->rightWing, depth + 1);
```

```
    for (int i = 0; i < depth; i++) printf("  ");
```

```
    printf("%s\n", getPlaneName(root->model));
```

```
    printAirTree(root->leftWing, depth + 1);
```

```
}
```

```
bool checkPlanesInRange(AirNode* root, int minIdx, int maxIdx) {
```

```
    if (!root) return true;
```

```
    // Если узел является листом
```

```
    if (!root->leftWing && !root->rightWing) {
```

```
        return (root->model >= minIdx && root->model <= maxIdx);
```

```
    }
```

```
    // Рекурсивно проверяем все ветви
```

```
    return checkPlanesInRange(root->leftWing, minIdx, maxIdx) &&
```

```
        checkPlanesInRange(root->rightWing, minIdx, maxIdx);
```

```
}
```

```
// Освобождение память
```

```
void clearAirTree(AirNode* root) {
```

```
    if (!root) return;
```

```
    clearAirTree(root->leftWing);
```

```
    clearAirTree(root->rightWing);
```

```
    free(root);
```

```
}
```

```
int main() {
```

```
    SetConsoleOutputCP(CP_UTF8);
```

```
    SetConsoleCP(CP_UTF8);
```

```
    AirNode* hangar = NULL;
```

```
    int choice, typeVal, rMin, rMax;
```

```

while (1) {

    printf("\n--- АВИАДИСПЕТЧЕР (Задание 7) ---\n");

    printf("1. Добавить самолет в ангар\n");
    printf("2. Показать структуру (дерево)\n");
    printf("3. Удалить самолет\n");
    printf("4. Проверить листья на диапазон (Вариант 7)\n");
    printf("5. Выход\n");
    printf("Выбор: ");

    if (scanf("%d", &choice) != 1) break;

    switch (choice) {

        case 1:

            printf("1-Cessna, 2-Beechcraft, 3-Antonov, 4-Sukhoi, 5-Airbus, 6-Boeing, 7-Concorde\n");
            printf("Введите ID типа: ");
            scanf("%d", &typeVal);
            hangar = insertPlane(hangar, (AircraftType)typeVal);
            break;

        case 2:

            if (!hangar) printf("Ангар пуст.\n");
            else printAirTree(hangar, 0);
            break;

        case 3:

            printf("Введите ID для списания: ");
            scanf("%d", &typeVal);
            hangar = removePlane(hangar, (AircraftType)typeVal);
            break;

        case 4:

            if (!hangar) {
                printf("Дерево не создано.\n");
            } else {
                printf("Задайте диапазон ID (минимум и максимум): ");
                scanf("%d %d", &rMin, &rMax);
                if (checkPlanesInRange(hangar, rMin, rMax))
                    printf("РЕЗУЛЬТАТ: Все листья входят в диапазон.\n");
                else
                    printf("РЕЗУЛЬТАТ: Найдены листья вне диапазона.\n");
            }
            break;
    }
}

```

```

case 5:

    clearAirTree(hangar);

    return 0;

default:

    printf("Неверный пункт.\n");

}

}

return 0;

}

```

9. **Дневник отладки** должен содержать дату и время сеансов отладки и основные события (ошибки в сценарии и программе, нестандартные ситуации) и краткие комментарии к ним. В дневнике отладки приводятся сведения об использовании других ЭВМ, существенном участии преподавателя и других лиц в написании и отладке программы.

№	Лаб. или дом.	Дата	Время	Событие	Действие по исправлению	Примечание

10. **Замечания автора** по существу работы: замечания отсутствуют.

11. **Выводы:** В ходе лабораторной работы были успешно освоены базовые принципы работы в командной строке ОС Unix и Python. Приобретены практические навыки по управлению файлами, архивации, поиску данных и автоматизации задач с помощью стандартных утилит и bash-скриптов и языка Python.

Подпись студента _____